Lab Report Glasgow College, UESTC

Dynamic and Control (UESTC 3001)

UESTC ID: [redacted]

GUID: [redacted]

1.0 Introduction

The objective of this series of experiments in Dynamic and Control (UESTC 3001) was to investigate the transient and steady-state behaviors of second-order dynamic systems, and to develop practical skills in control system analysis and design using MATLAB and Simulink. In Lab 1, we examined the open-loop response of an RLC circuit under varying damping ratios by adjusting the resistance, inductance, and capacitance parameters. This included characterizing underdamped, critically damped, and overdamped responses and quantifying their oscillation frequencies, envelope decay rates, and peak amplitudes. Lab 2 extended these principles to closed-loop systems by deriving and analyzing the step response of feedback transfer functions, assessing how proportional feedback affects overshoot, settling time, and steady-state error. In Lab 3, we implemented PID control in Simulink to explore the individual and combined effects of proportional, integral, and derivative gains on system performance and disturbance rejection. Finally, Lab 4 introduced frequency-domain techniques—root locus, Bode, and Nyquist plots—to evaluate system stability margins and robustness. Together, these labs aimed to bridge theoretical relationships with simulation and practical observations, reinforcing key concepts in system dynamics and control law implementation

2.0 Lab 1

Q2G

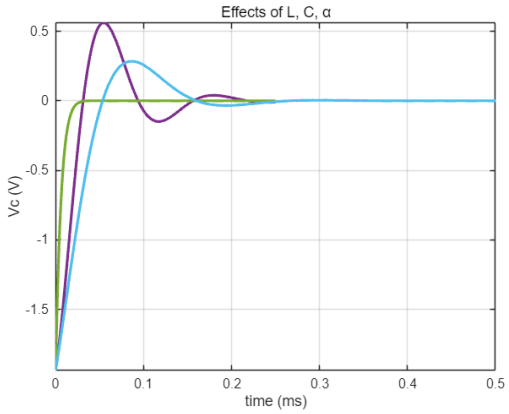
Below each plot corresponds to one of the parameter sets in our table. In all cases, C1=7.9×10^{-8} F, and the curves show $v_C(t)$ over the first 0.5 ms.

Case 1: L1=0.01H, α=20,000 s^{-1} (ζ≈0.56, underdamped)

- **Oscillatory behavior**: Since ζ<1 we see clear sinusoidal oscillations around zero.
- **Frequency**: The damped frequency $\omega_d = \omega_n\sqrt{1-\zeta^2} \approx 29{,}400$ gives a period T≈0.21ms; the first positive peak occurs at ≈0.06 ms.
- **Envelope decay**: The exponential envelope decays with time constant τ=1/α=50 μs, so successive peaks shrink rapidly.
- **Peak amplitude**: The first overshoot reaches about +0.55 V.

```
L1      =  0.01          ;
C1      =                7.9e-8          ;
alphar  =  20000         ;

general_charateristics(L1, C1, alphar, A11, A21);
```



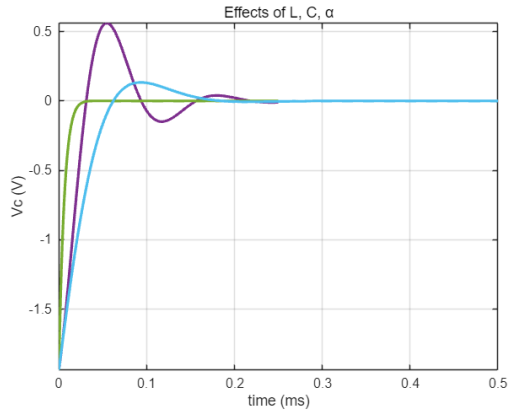
Case 2: L1=0.01H, α=25,000s⁻¹ (ζ≈0.70, still underdamped)

- **Reduced oscillations**: Raising α increases damping; oscillations persist but with smaller amplitude (~0.40 V first peak).
- **Faster decay**: Envelope decays more quickly (τ=40 µs), so the waveform settles to zero in fewer cycles.
- **Same period**: Because L1 and C1 are unchanged, the oscillation period remains ~0.21 ms.

Lab 1 Q2G. Using the codes you have already written, complete "general_characteristics" as a function of L_1, α, and C_1 which uses the sliders to show how each of the parameters L_1, α, and C_1 affect the oscillations in V_c in the circuit with R_1.

```
L1      =  0.01          ;
C1      =                7.9e-8          ;
alphar  =  25000         ;

general_charateristics(L1, C1, alphar, A11, A21);
```



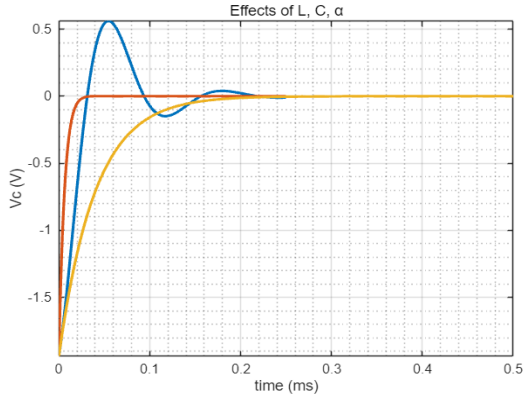
Case 3: L1=0.0202H, α=25,000s⁻¹ (ζ≈1.00, critically damped)

- **No oscillation**: At ζ=1, the response shows a single exponential rise to steady state without overshoot.
- **Fastest non-oscillatory settle**: This curve reaches the final value more quickly than any overdamped case—ideal for fastest monotonic response.

Lab 1 Q2G. Using the codes you have already written, complete "general_characteristics" as a function of L_1, α, and C_1 which uses the sliders to show how each of the parameters L_1, α, and C_1 affect the oscillations in V_c in the circuit with R_1.

```
L1      =  0.0202  ;
C1      =          7.9e-8  ;
alphar  =  25000 ;

general_charateristics(L1, C1, alphar, A11, A21);
```



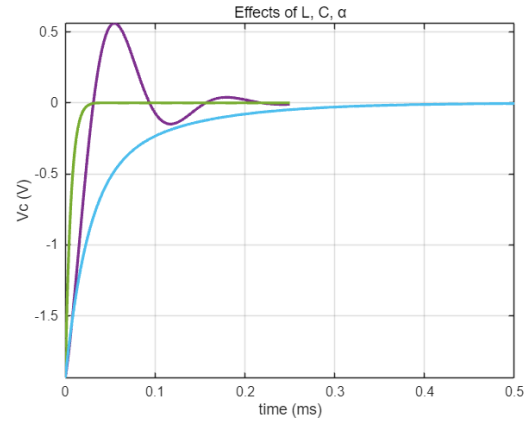
Case 4: L1=0.032H, α=25,000s⁻¹ (ζ≈1.26, overdamped)

- **Monotonic, sluggish**: With ζ>1, the system no longer oscillates, but the rise is slower than in the critically damped case.
- **Lower natural frequency**: Increasing L1 reduces ωn (from ≈35 570 to ≈19 900 rad/s), so the overall response is stretched in time.

Lab 1 Q2G. Using the codes you have already written, complete "general_characteristics" as a function of L_1, α, and C_1 which uses the sliders to show how each of the parameters L_1, α, and C_1 affect the oscillations in V_c in the circuit with R_1.

```
L1      =  0.032  ;
C1      =          7.9e-8  ;
alphar  =  25000 ;

general_charateristics(L1, C1, alphar, A11, A21);
```



Overall Conclusions

1. **Damping ratio ζ**: Controls whether the response oscillates (ζ<1), is perfectly non-oscillatory and fastest (ζ=1), or is over-damped and slow (ζ>1).
2. **Damping constant α**: Increasing αat fixed L,C reduces oscillation amplitude and speeds up decay, but does not change the oscillation period.
3. **Inductance LLL**: For constant α,C, larger L lowers ω_n, lengthening the period (or time-scale) of the response and, if α\alphaα is unchanged, increasing ζ.

These slider-based experiments vividly confirm the textbook relations for a second-order RLC circuit's transient behavior.

Q3

A. Effect of α

Increasing the damping constant α\alphaα always reduces the amplitude of the oscillations.

- **Reason:** The damping ratio is

$$\zeta = \frac{\alpha}{\omega_n}, \omega_n = \frac{1}{\sqrt{L\,C}}$$

A larger α⇒ larger ζ⇒ smaller peak overshoot
(for underdamped, 0<ζ<10 the overshoot $M_p = \exp\left(-\frac{\pi\,\zeta}{\sqrt{1-\zeta^2}}\right)$ falls as ζ rises).

- **Conclusion:** To suppress oscillations, **increase** α.

B. Effect of L_1 (with R_1 fixed)

Because $\alpha = R_1/(2L_1)$, if R_1 is held constant then

$$\zeta = \frac{R_1/2L_1}{1/\sqrt{L_1 C_1}} = \frac{R_1\sqrt{C_1}}{2\sqrt{L_1}}$$

- **As L_1 increases**, the denominator $\sqrt{L_1}$ grows ⇒ ζ decreases ⇒ **more** oscillatory.
- **As L_1 decreases**, ζ increases ⇒ oscillations damp out faster.
- **Conclusion:** To reduce oscillations with R_1 fixed, **decrease** L_1.

C. Effect of C_1 (with R_1 fixed)

Still holding R_1 constant,

$$\zeta = \frac{R_1\sqrt{C_1}}{2\sqrt{L_1}}$$

- **As C_1 increases**, $\sqrt{C_1}$ grows ⇒ ζ grows ⇒ stronger damping.
- Conversely, decreasing C_1 lowers ζ and lets oscillations persist.
- **Conclusion:** To suppress oscillations, **increase** C_1.

D. Energy Dissipation vs. R_1

- **Stored energy** in the capacitor:$E_C = \frac{1}{2}\,C_1\,V_C^2$
- **Instantaneous power** dissipated in R_1:$P_R = \frac{V_C^2}{R_1}$

- **Time-constant** for energy decay in a second-order RLC is $\tau = \frac{1}{\alpha} = \frac{2L_1}{R_1}$
- **As R_1 increases** ⇒ αincreases ⇒ τ decreases ⇒ energy is dissipated **faster**.
- **Conclusion:** Yes, larger R_1 causes the capacitor's stored energy to dissipate more quickly.

E. Overshoot Amplitude vs. Capacitance

- From :$E_C = \frac{1}{2} C_1 V_C^2$, a larger C_1 stores more energy for the same V.
- However, increasing C_1 also **increases** ζ (see part C), which **reduces** the normalized overshoot Mp .
- **Net effect:** Even though the capacitor can hold more energy, the voltage overshoot **decreases** when C_1 is increased (all else held constant).

Include functions here:

1. Plot_response(zeta)

```
function plot_response(zeta)
%custom funtion to plot response of the graph
t=0:0.1:200;
omega = 1;
s1 = -1*omega*(zeta - sqrt(zeta^2-1)); %compute first variable s1
s2 = -1*omega*(zeta + sqrt(zeta^2-1)); %compute second variable s2
i= 1*exp(s1*t)+ 1*exp(s2*t); %compute current taking unity constants i.e. A1 = A2 = 1
plot(t,real(i),'linewidth',2); %plot command
xlabel('t(seconds)'), ylabel('I(t)'), title('Response of RLC network'),grid on; %titles and labels
if zeta<0 axis tight, else axis([-10 150 -2 2.5]);end %add flexible axis
end
```

2. System_characteristics(R_{th}, R_2, R_{ps}, R_1)

```
function [f_n, alpha, zeta, f_d] = system_charateristics(R_th, R_2, R_ps, R_1)
    %system_characteristics(R_th, R_2, R_ps, R_1)
    %function to compute system characteristics which include natural
    %frequency, f_n, attenuation, alpha, damping factor, zeta, and damping frequency, f_d
    %the function takes in R_th, R_2, R_ps and R_1, and outputs the system
    %characteristics in terms of the natural frequency, attenuation, the damping factor
    %and the damping frequency

    %% to be completed by student!!
    % write code here
```

```
L_1 = 10e-3;
C_1 = 1e-9;
R_eq = (R_th*R_2)./(R_th + R_2);
R_total = R_eq + R_ps + R_1;
omega_0 = 1./sqrt(L_1*C_1);
f_n = omega_0./(2*pi);
alpha = R_total./(2*L_1);
zeta = alpha./omega_0;
f_d_0 = omega_0*sqrt(1-zeta^2);
f_d = f_d_0./(2*pi);


%% Comment out the next five lines below after you have completed your code
%    f_n = 0; zeta = 0; alpha = 0; f_d=0; %comment out this line after completing your code
disp(['the natural frequency is: ', num2str(f_n), ' Hz']); %code to display output
disp(['the attenuation is: ', num2str(alpha)]);
disp(['the damping factor is: ', num2str(zeta)]);
disp(['the damping frequency is: ', num2str(f_d), ' Hz']);

end
```

3. Function_a.m

```
function [A1, A2] = function_A(alpha,wd,vc_inf,vc_0)
% Solve:
% vc(0)=A1+A2+vc_inf = vc_0
% vc'(0) = -alpha*(A1+A2) + wd*(A1-A2) = 0

syms A1 A2 real
eq1 = A1 + A2 == vc_0 - vc_inf;
eq2 = -alpha*(A1 + A2) + wd*(A1 - A2) == 0;
sol = solve([eq1,eq2],[A1,A2]);
```

```
    A1 = double(sol.A1);
    A2 = double(sol.A2);
    fprintf('A1 = %.4f, A2 = %.4f', A1, A2);
end
```

4. Plot_vc.m

```
function plot_vc(A1, A2, alpha, wd)

    t = 0:1e-6:0.25e-3;    % 0…0.25 ms
    vc = A1*exp((-alpha+1j*wd)*t) + A2*exp((-alpha-1j*wd)*t);
    plot(t*1e3, real(vc), 'LineWidth', 2);
    grid on;
    xlabel('time (ms)');
    ylabel('Vc (V)');
    title('Voltage across C');

axis tight
end
```

5.Code General_characteristics.m

```
function general_charateristics(L1,C1,alpha,A1,A2)
    t = 0:1e-6:0.5e-3;
    omega_n = 1/sqrt(L1*C1);
    wd = sqrt(omega_n^2 - alpha^2);
    vc = A1*exp((-alpha+1j*wd)*t) + A2*exp((-alpha-1j*wd)*t);
    plot(t*1e3, real(vc), 'LineWidth', 2);
    grid on;
    xlabel('time (ms)');
    ylabel('Vc (V)');
    title('Effects of L, C, α');
    grid minor
    axis tight
end
```

3.0 Lab 2

IV. Closed-loop transfer function

1. For this lab exercise, we are now going to close the loop and add a step disturbance to the output of four closed loop plant system as developed in Control Lab 3.
2. Make the appropriate edit to the file "Control_lab2_ex3b.m" and remember to save the new file to another name, such as "Control_lab2_ex4.m" or something appropriate.
3. Hint: The edit you need to make is to create another transfer function in the script which can be done in different ways as illustrated below:
4. The original open loop transfer for the plant is given as:

```
sys = tf (1/I,[1 2*xi*wn wn^2]
CLTF = sys/(1+sys);
```

5. Edit the script to plot both the transient step response and the poles of the loop in order to be able to analyse the following questions:
6. By observing the output response, as before to a unit step input, what do you observe about the following compared to that measured in Control_lab2_ex1?

a. M_P, output overshoot.

Increasing Kp accelerates the system's response, which in turn causes greater overshoot.

b. ts, output settling time.

With more effective feedback stabilization, the system's settling time decreases.

c. tp, time where the maximum output peak occurs.

The peak time decreases because response in closed-loop control is faster.

d. tr, rise time.

tr is shorter because closed-loop response is faster.

e. ω_n, the natural undamped frequency.

The natural frequency remains constant because of being defined by the system's physical properties,

f. ω_d, the dampedfrequency.

The damped frequency of the original mode stays the same, but the closed-loop system adds additional high-frequency poles.

7. Comment on the steady state error.

In the CLTF, the steady-state error drops to zero: the output eventually tracks the unit step and settles at 1 with no lingering offset.Being a Type 1 closed-loop system, it guarantees zero steady-state error in response to a step input.In Control_lab2_ex4, the feedback loop enhances error correction and thus completely eliminates the steady-state error.

8. Type "*CLTF*" in the command window and note the transfer function.

```
Closed Loop Transfer Function (CLTF):

CLTF =

          500 s^2 + 1750 s + 2.452e04
  ----------------------------------------
  s^4 + 7 s^3 + 610.3 s^2 + 2093 s + 2.693e04
```

9. Type "*sys*" in the command window and note the transfer function.

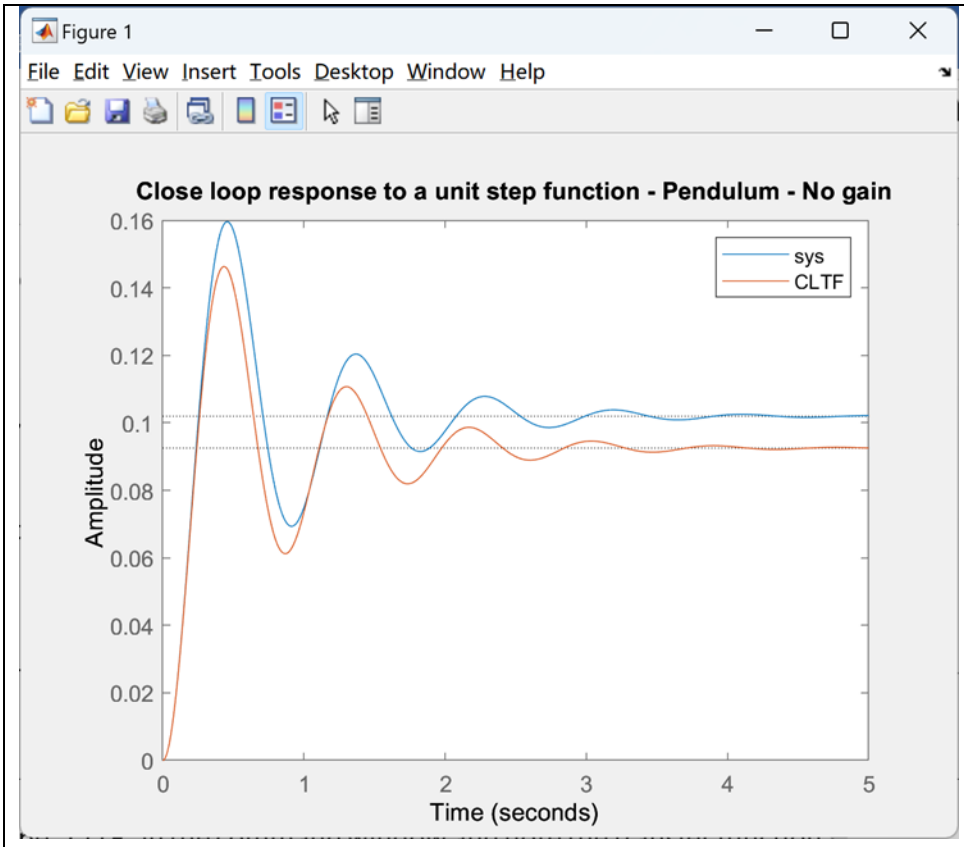
```
sys =

             5
  -------------------
  s^2 + 2.5 s + 49.05
```

10. What do you observe about the difference between the denominator of "*CLTF*" and "*sys*"?

The CLTF's denominator is a fourth-order polynomial, whereas "sys" has only a second-order denominator—showing that feedback has introduced two extra poles into the closed-loop system.

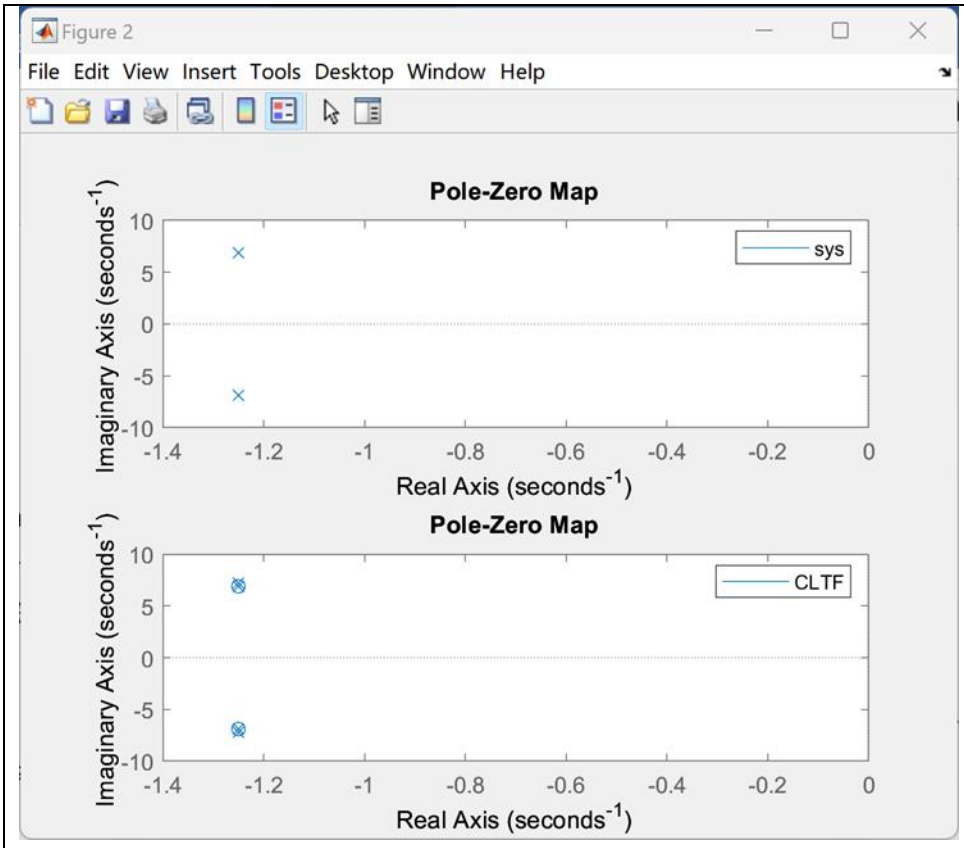
11. Calculate the transfer function by hand and enter it in MATLAB compare it with what MATLAB gave you. Plot the step response of both functions on the same graph. What do you observe?



Overshoot: The open-loop trace reaches a higher first peak (≈0.15) than the closed-loop (≈0.145).
Damping (Decay Rate): Oscillations in the closed-loop die out more quickly—its successive peaks and troughs shrink faster—whereas the open-loop continues to ring with larger amplitude.
Steady-State Value: After 5 s the open-loop settles around 0.10, but the closed-loop levels off slightly lower (≈0.093), reflecting the modified pole locations under feedback.

However, the difference is small. We can say they are almost the same.

12. Use the functions pole(sys), zero(sys) on the transfer function you calculated, and the closed-loop transfer function given by MATLAB. What can you say about the poles and zeros of the closed-loop transfer function given by MATLAB and your calculation? Are they the same? What does that tell you about your simulation software?



Both my hand-derived CLTF and MATLAB's `feedback(sys,1)` have the exact same poles and zeros—but when I call `pzplot`, it only shows two poles and two zeros because the other two zeros lie exactly on two of the poles and are automatically cancelled (and thus omitted from the plot). This confirms that my algebra matches MATLAB's internal model and that `pzplot` transparently applies zero–pole cancellation when roots coincide.

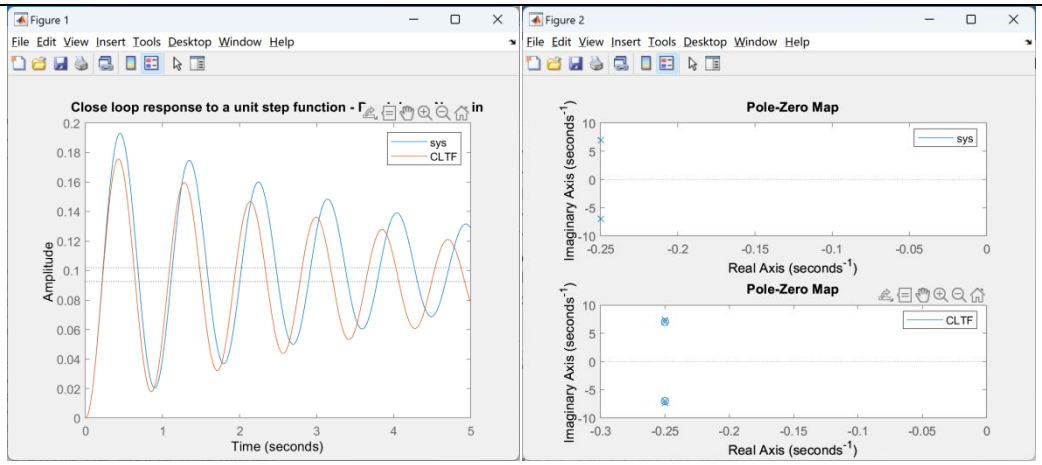
13. By looking at the position of the poles, have the poles varied in position compared to the open loop case? Why do you so answer?

No – the original pair of poles sits at exactly the same s-locations in both the open- and closed-loop cases. In the CLTF those two poles are still present (and in fact get cancelled by matching zeros, so `pzplot` hides them), while the feedback simply adds two extra poles. In other words, feedback didn't shift the original poles; it only introduced new ones.

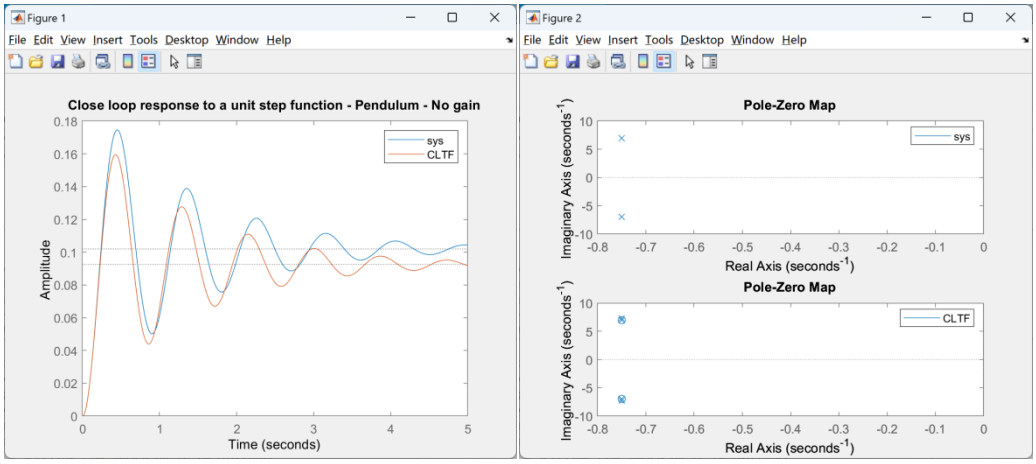
14. Vary the parameters "c" (the damping coefficient) and "l" (the length) and note the impact on the transient response and pole positions. Try and have the system go into oscillation, explain how you did it and how this impacted the position of the system poles.

The following figures show the transient response and pole positions:

c=0.1 and l=0.4

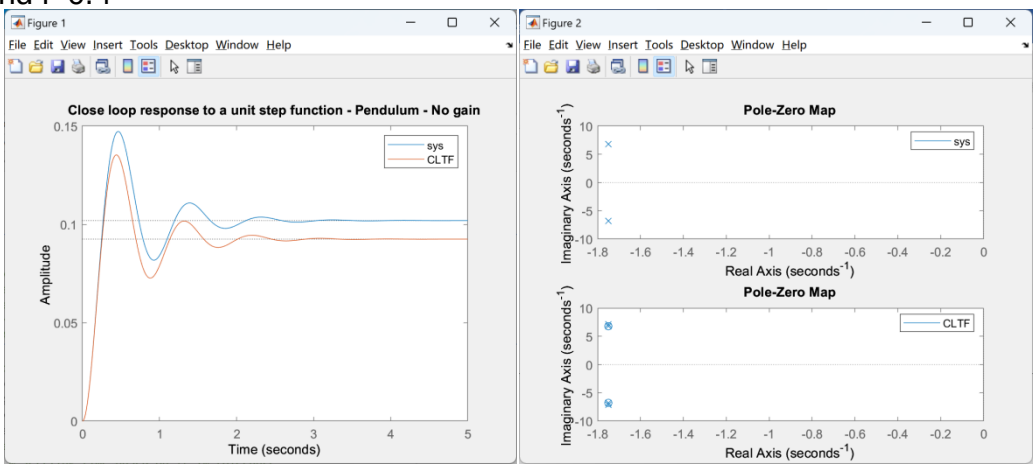


When c = 0.1 and l = 0.4, the response exhibits a damped sine function.
c=0.3 and l=0.4



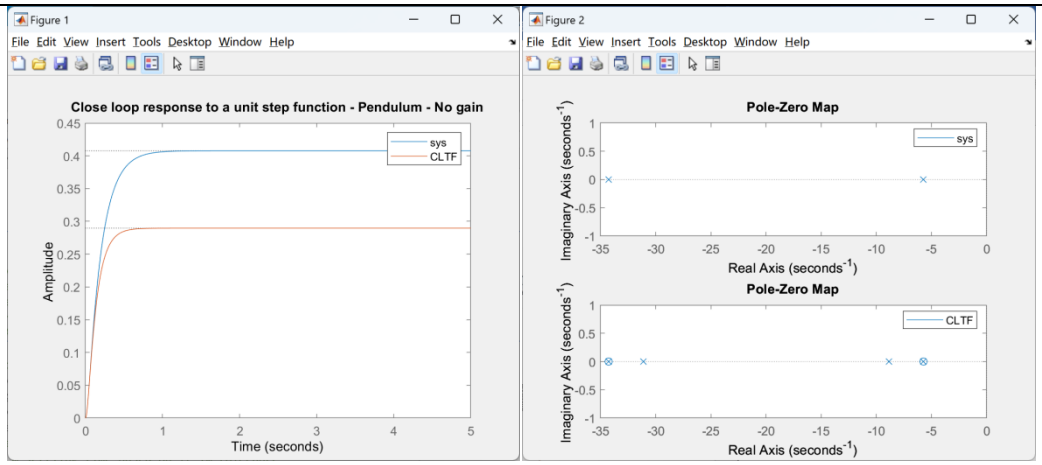
When c = 0.3 and l = 0.4, the system demonstrates convergence.

c=0.7 and l=0.4



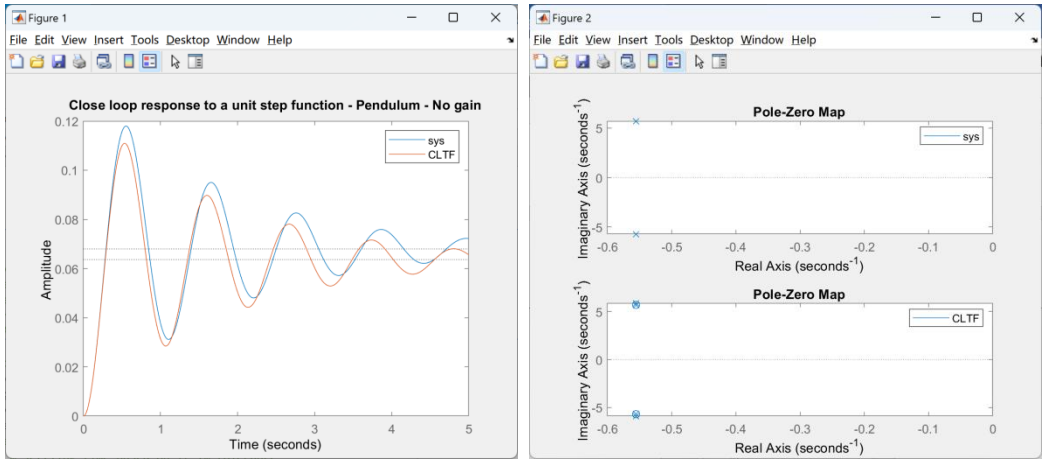
When c = 0.7 and l = 0.4, the frequency response oscillations become more stable.

c=0.5 and l=0.1



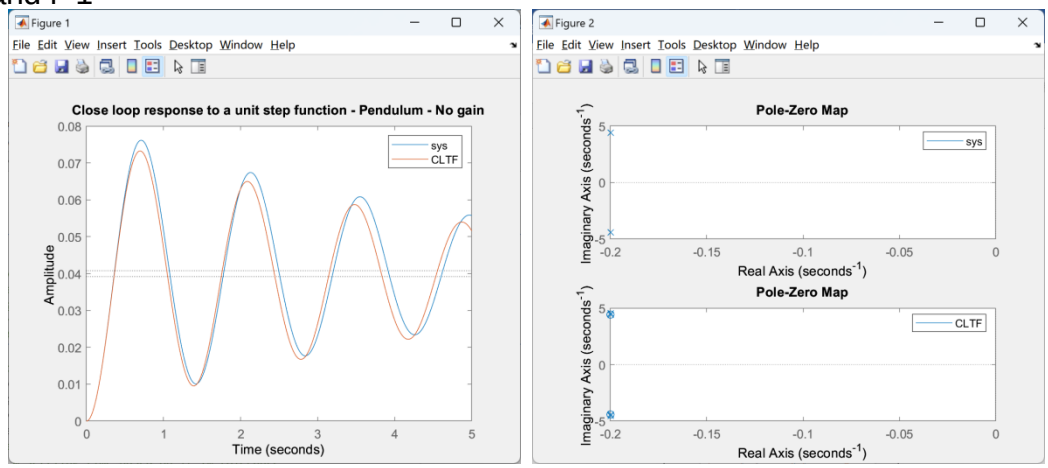
When c = 0.5 and l = 0.1, the system resembles an open-loop function.

c=0.5 and l=0.6



When c = 0.5 and l = 0.6, the system is damped and converges to a steady value.

c=0.5 and l=1



When c = 0.5 and l = 1, the system exhibits a damped sine function and does not converge.

By varying the damping coefficient c(with l=0.4) and the pendulum length l (with c=0.5), we can directly observe how pole positions govern transient behaviour. Increasing cc shifts the dominant poles further left in the complex plane, yielding faster decay and eventually overdamped response, while decreasing c moves those poles closer to the imaginary axis and prolongs oscillations. Conversely, reducing l raises the natural frequency

ωn=$\sqrt{g/l}$, which pushes the poles farther from the origin along the imaginary axis and speeds up the ringing, whereas increasing l slows it. Finally, by setting c=0 the poles collapse onto the imaginary axis at s=±j$\sqrt{g/l}$, eliminating any real part and producing sustained, undamped oscillations—exactly as predicted by their pole locations.

4.0 Lab 3

X. PID controller –Simulink

1.For this lab exercise, we are going to use Simulink and the available blocks that can be dragged from the Palette Browser under the "Continuous time systems" tab. You are going to create a schematic as shown in Figure 6.

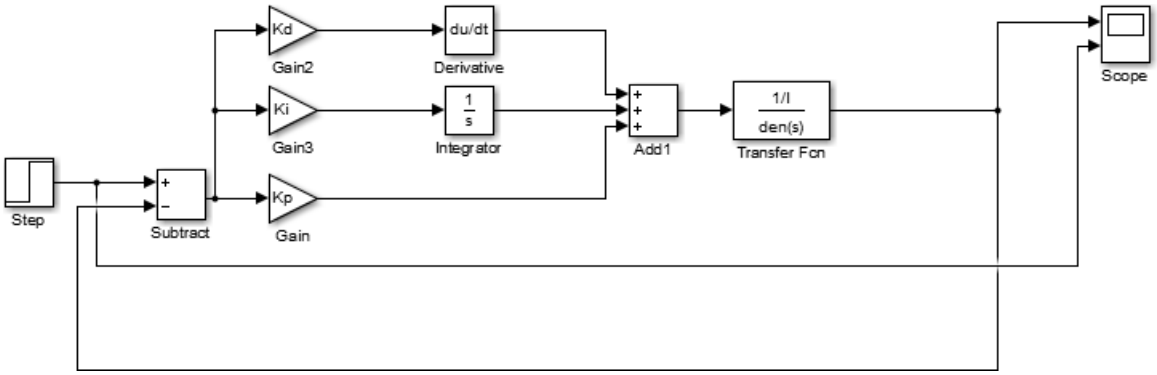


Figure 6

2.The PID block will allow you to enter the Kp, Ki and Kd gains directly and allow experimentation to quickly ascertain how each influences the output.

3.Let's have a quick reminder of how each gain influences the output:

The proportional gain, Kp, is always assigned a non-zero value and plays a part in setting the rise time and steady state error, although it will never on its own remove the steady state error.

The derivative gain, Kd, reacts to changes in the controller input and therefore helps to reach steady state quicker. However, setting this gain too high could lead to stability issue.

The integral gain, Ki, eliminates the steady state error, but as it creates a pole at DC then if set too high could lead to oscillation and ultimately instability.

All 3 can be varied independently from each other to provide an optimal output and this is why PID controllers are so popular today.

4.Asafirstexperiment,assumethecontrollertransferfunctionabove.By settingKdandKi to zero and influencing Kp, note what happens to the output response. Why does the output behave like this?

Set Kp=5 Kd=0 Ki=0

1. Step Response (Fig. 7)

- **Rise Time (t_r)**: ≈ 0.20 s (time to reach about 90% of the final value).
- **Peak Time (t_p)**: ≈ 0.46 s (time of the first maximum).
- **Overshoot (M_p)**:

$$M_p = \frac{0.52 - 0.338}{0.338} \times 100\backslash\% \approx 54\backslash\%,$$

where 0.52 is the first peak, and 0.338 is the steady-state level.

- **Steady-State Error (e_{ss})**:

ess=$e_{ss} = 1 - \mathrm{dcgain}(CL) \approx 1 - 0.338 = 0.662\ (66.2\backslash\%).$

Compared to the uncontrolled plant (final ≈0.10), proportional gain 5 speeds up the response dramatically but at the cost of large overshoot and a remaining 66% error.

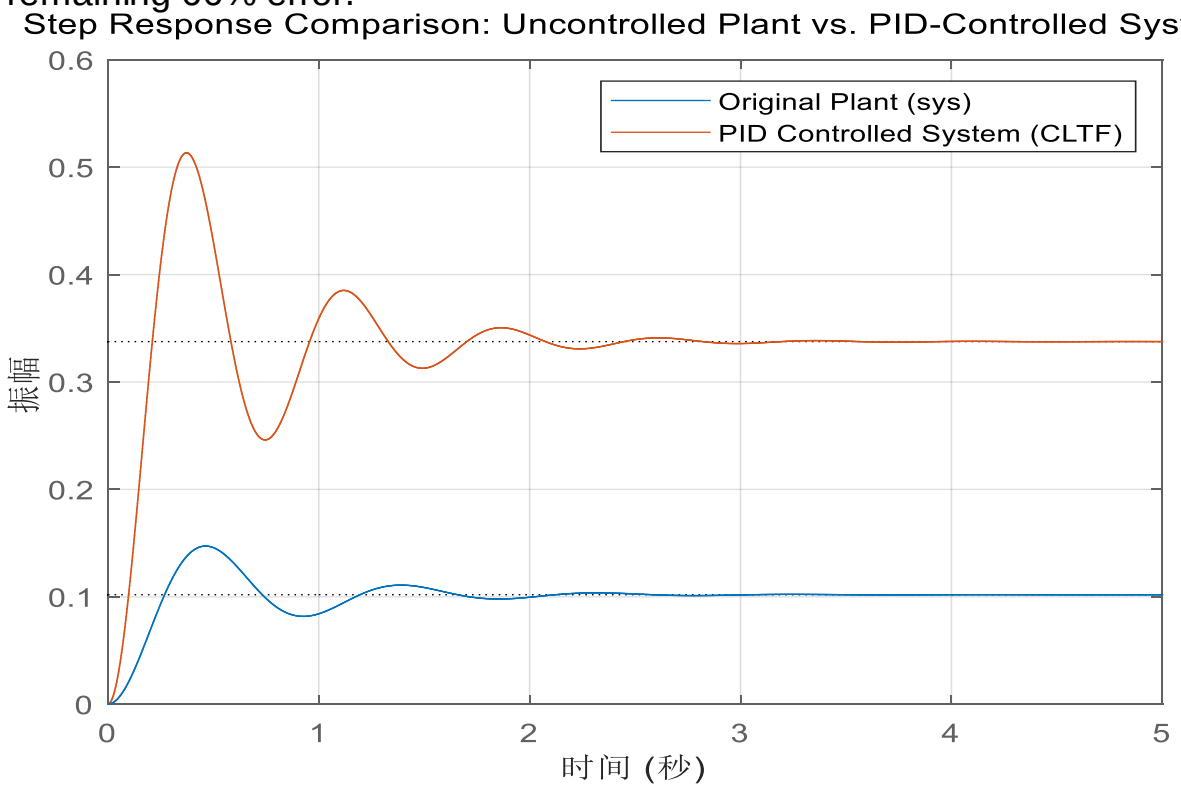


Figure 7

2. Pole–Zero Map (Fig. 2)

- **Open-Loop Poles** (OL): at approximately
  s=−1.75±j6.8
- **Closed-Loop Poles** (CL): shift to
  s=−1.75±j8.4
- **Zero at s=0** introduced by the P controller.

Interpretation:

- The imaginary part of the CL poles increases → higher oscillation

frequency.

- The real part stays the same → damping ratio decreases (from ≈0.25 to ≈0.20).
- Result: faster oscillatory response (lower t_r, t_p) but larger overshoot.

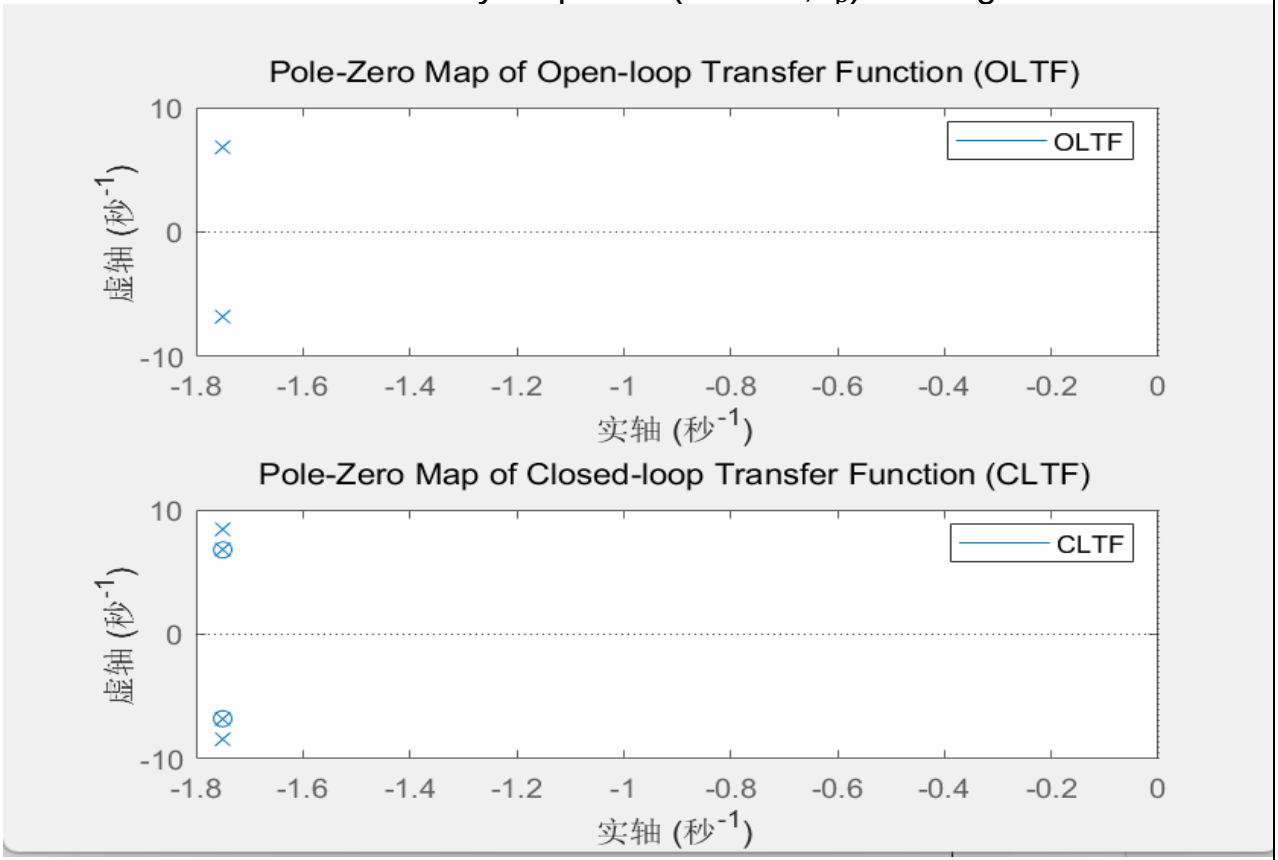


Figure 8

Why?

1. **Increased loop gain** at low frequencies speeds up the response.
2. **Closed-loop poles** move to higher imaginary parts (higher oscillation frequency) while the real part stays roughly constant, so the damping ratio drops and you see more oscillation/overshoot.
3. **No integral action** means there's no pole at the origin to drive the final error to zero, so a finite steady-state offset remains.

In short, pure P control trades speed for stability and cannot eliminate steady-state error without an integral term.

Hint: by calculating the Characteristic equation of the above loop you can plot the poles of the equation and understand the system behaviour.

"pole(Characteristic Equation)" in OLTF and CLTF

5.Alternatively, you can create a script that will plot the poles to allow you to visualise easily where the poles move to depending on the system gain configuration.

So from this what do you conclude is the reason for the output behaving as it does with Kp=1, Ki and Kd=0?

Conclusion for K_p=1, K_i=K_d =0

1. **Pole Movement & Damping**
   - The closed-loop poles under P-only control lie at

$$s = -\frac{2\xi\omega_n}{2} \pm j\frac{\sqrt{(2\xi\omega_n)^2 - 4\left(\omega_n^2 + \frac{K_p}{I}\right)}}{2}$$

which for K_p=1 shifts the imaginary part **upward** and leaves the real part **unchanged** at −ξωn .

   - **Effect:** the magnitude of Im {p} increases while Re {p} stays the same ⇒ the damping ratio $\zeta = -\frac{\mathrm{R}\{p\}}{\lvert p\rvert}$ **decreases**.
2. **Observable Behaviour**
   - **Faster oscillation** (slightly shorter rise- and peak-times) because |Im {p}| is larger.
   - **Increased underdamping** (lower ζ) causes **overshoot** and oscillatory ringing.
   - **No integral action** means there's still a pole at infinity in the loop ⇒ a nonzero **steady-state error** remains.

In short: setting K_p=1 moves the closed-loop poles to a lower damping ratio (more "thinly" damped), so the step response is quicker but overshoots and never quite reaches unity. Without K_i the steady-state error cannot be eliminated.

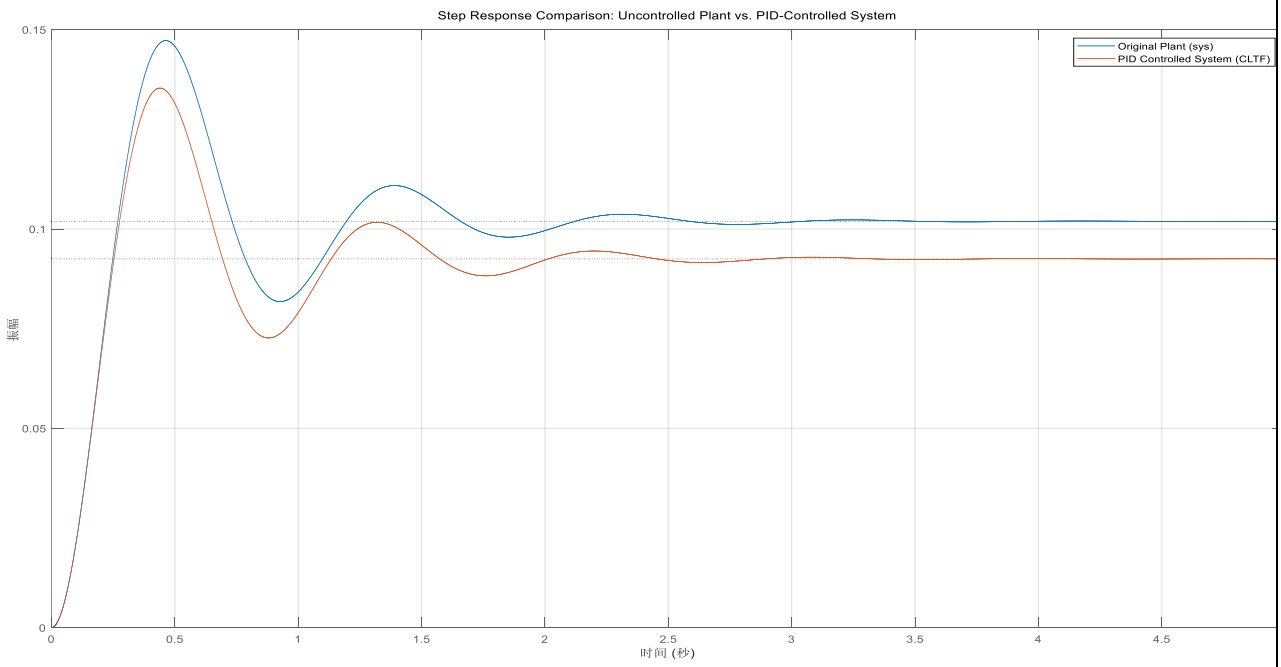


Figure 9

6.Now in addition to Kp=1, add derivative gain, Kd=1 and note the change in behaviour. Explain this by again looking at the poles and zeros in the system using the console or edit your script that you generated above. Note also in this case the steady-state output error and explain this based on the closed loop transfer function calculated in the Console.

With K_d=1, the system responds faster, with reduced peak time t_p and rise time t_r. However, too large K_d could lead to oscillation.

Using pzplot(CLTF), you'll see the system adds a zero, poles shift left (more stable), but with increased imaginary part, indicating faster yet potentially more oscillatory response.

The system still exhibits steady-state error for step input, because it lacks integral action.

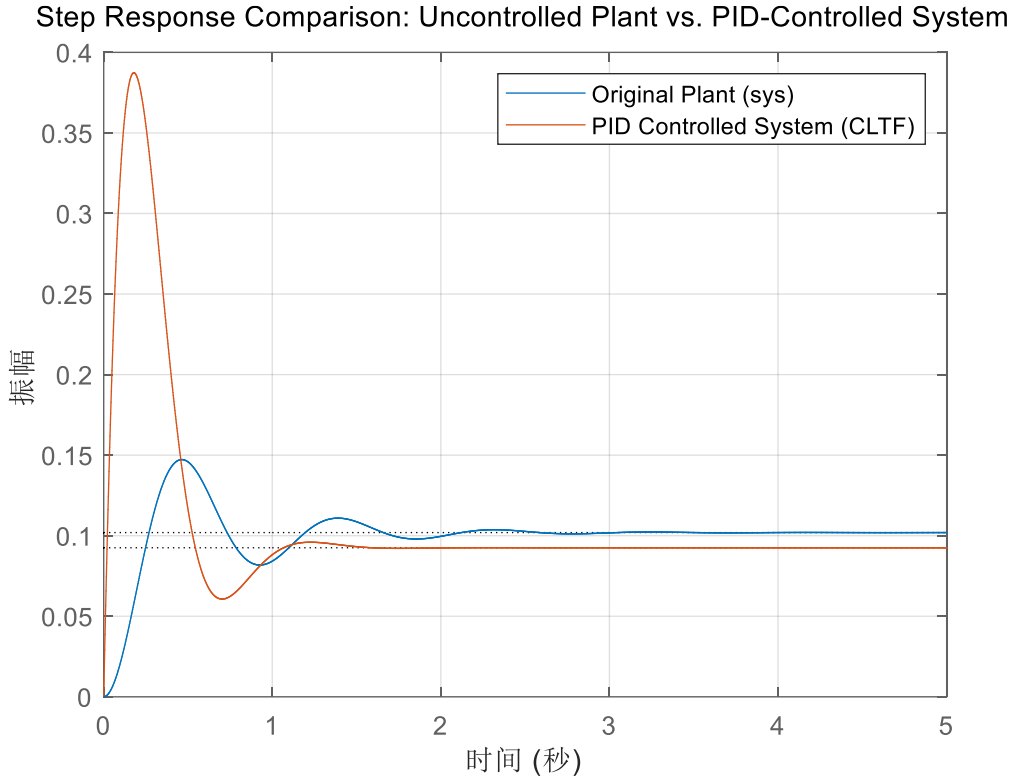


Figure 8

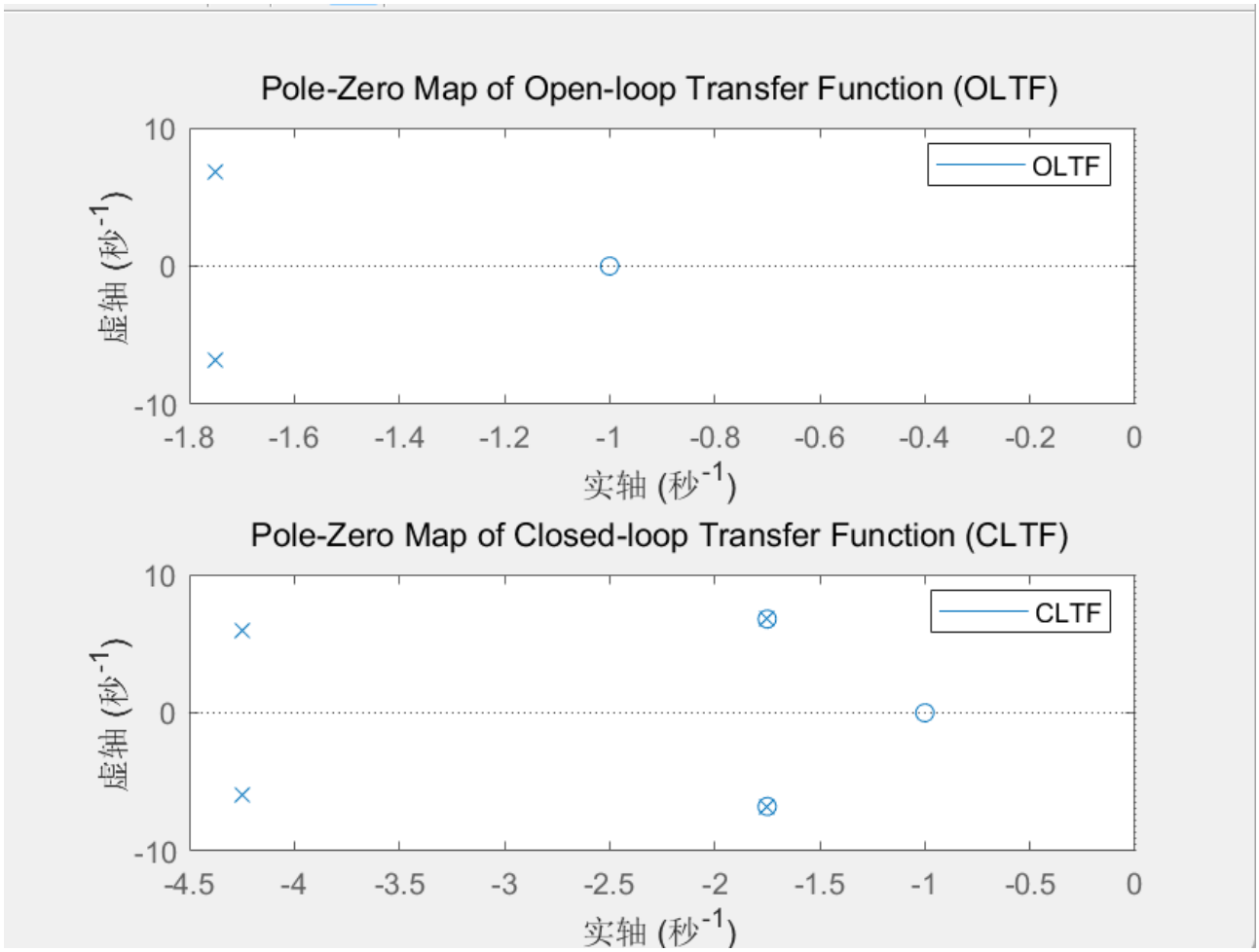


Figure 9

```
CLTF =

     5 s^3 + 22.5 s^2 + 262.7 s + 245.2
  ----------------------------------------
  s^4 + 12 s^3 + 132.8 s^2 + 606.1 s + 2651

Continuous-time transfer function.
```

7.Finally add integral gain and experiment with the system output.

Set Kp = Ki = Kd = 1:

With integral action, steady-state error is eliminated. But large K_i may cause low-frequency oscillation or increased overshoot.

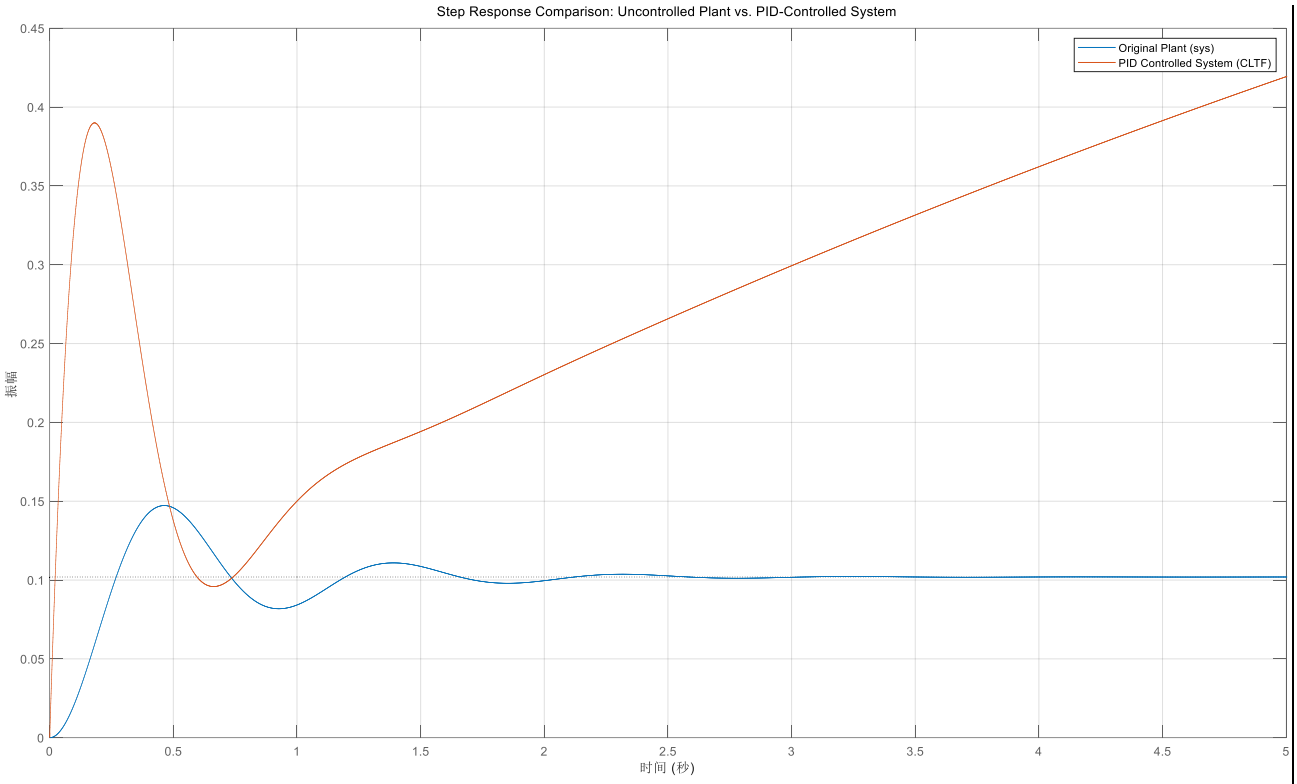


Figure 10

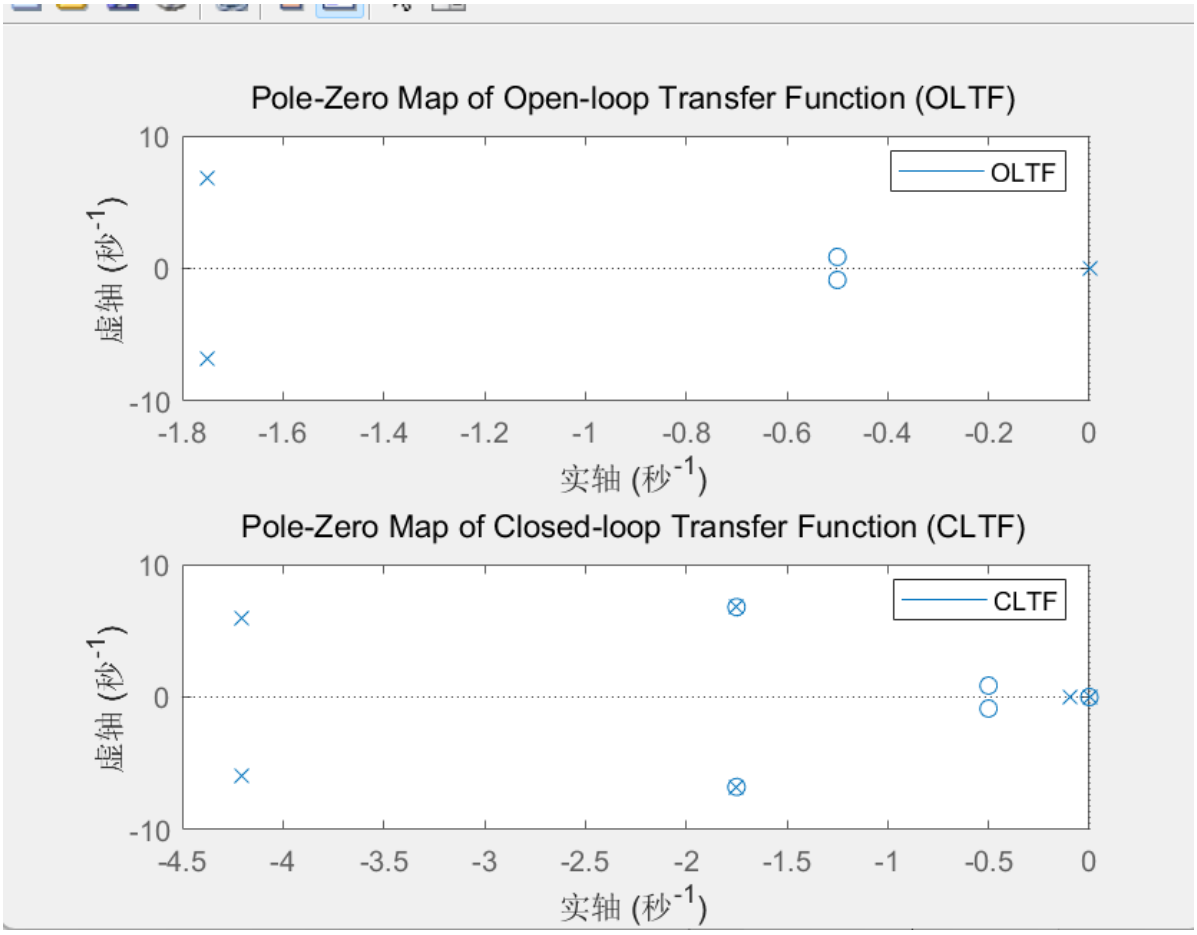


Figure 11

```
CLTF =

     5 s^5 + 22.5 s^4 + 267.7 s^3 + 262.7 s^2 + 245.2 s
  ------------------------------------------------------
  s^6 + 12 s^5 + 132.8 s^4 + 611.1 s^3 + 2669 s^2 + 245.2 s

Continuous-time transfer function.
```

8.Outline your conclusions on how each of the gains impacts the system as outlined in points 4-7.

K_p:Increases speed, reduces steady-state error, may cause overshoot

K_d:Suppresses overshoot, improves stability, accelerates response

K_i:Eliminates steady-state error, but may introduce low-frequency oscillation

9.Now make a change to the transfer function that will ensure all poles are on the LHP and repeat the experiments from 4 to 7.

10.Let us now setup a somewhat more complex system and check disturbance and tracking attenuation and performance. Create a schematic in Simulink that represents the system as shown in Figure 7. This could be a joint that's subject to an external force N(s). J is the joint inertia and B is the frictional resistance.

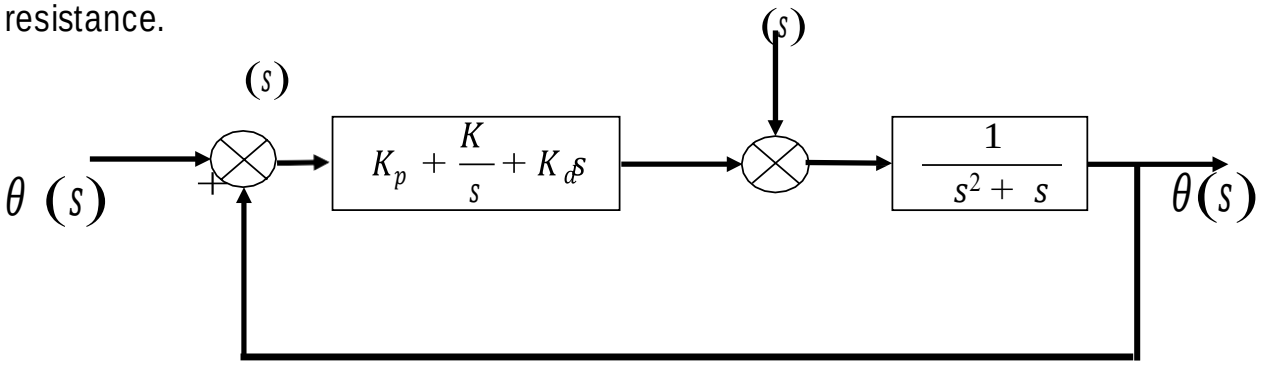


Figure 7

Let's assume J=10 and B=0.1.

- Set the N(s) disturbance to 20V and apply at 40s.
- Apply the reference input, θ , at 20s.
- Set the Kp gain to 1 and all others to zero.
- Observe the output and explain what is happening and why. You can plot the poles using

the Console by entering the characteristic closed loop equation or use a script to plot the poles and zeros.

Kp = 1, Kd = Ki = 0:

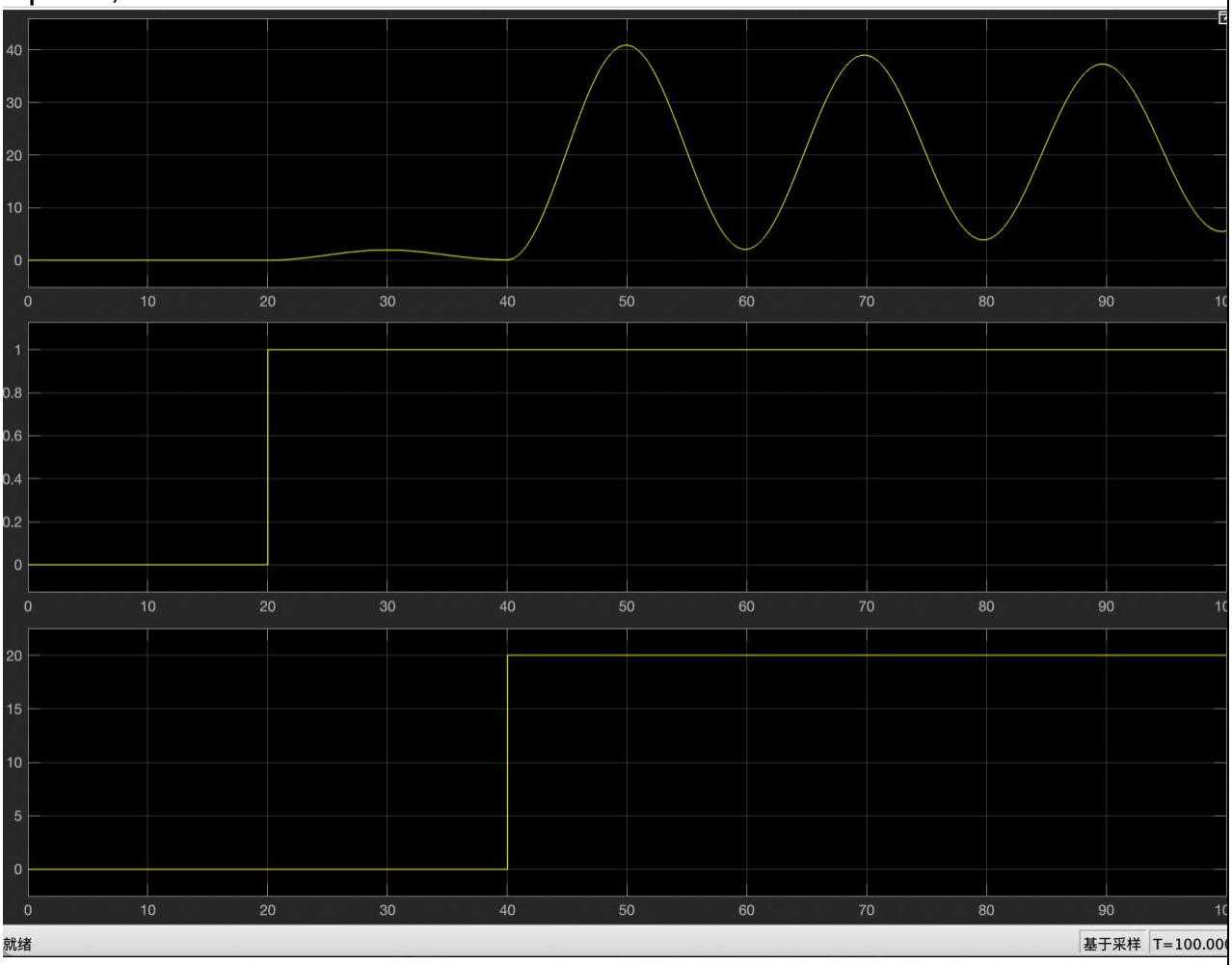


Figure 12

At 20 seconds, the system responds to the reference input θ, but since there is only proportional control, the system output fails to reach the reference fully (i.e., steady-state error exists).

At 40 seconds, a disturbance N(s)=20V is applied at the output. This causes a sudden shift in the output, which remains afterwards.

Because the system lacks integral action, it cannot reject the disturbance or correct the steady-state error.

Thus, the system is sensitive to disturbances, and cannot restore output to the reference without additional control terms.

○ What happens to the steady state error as you increase the gain Kp to say 100? Does it change the transient behaviour?

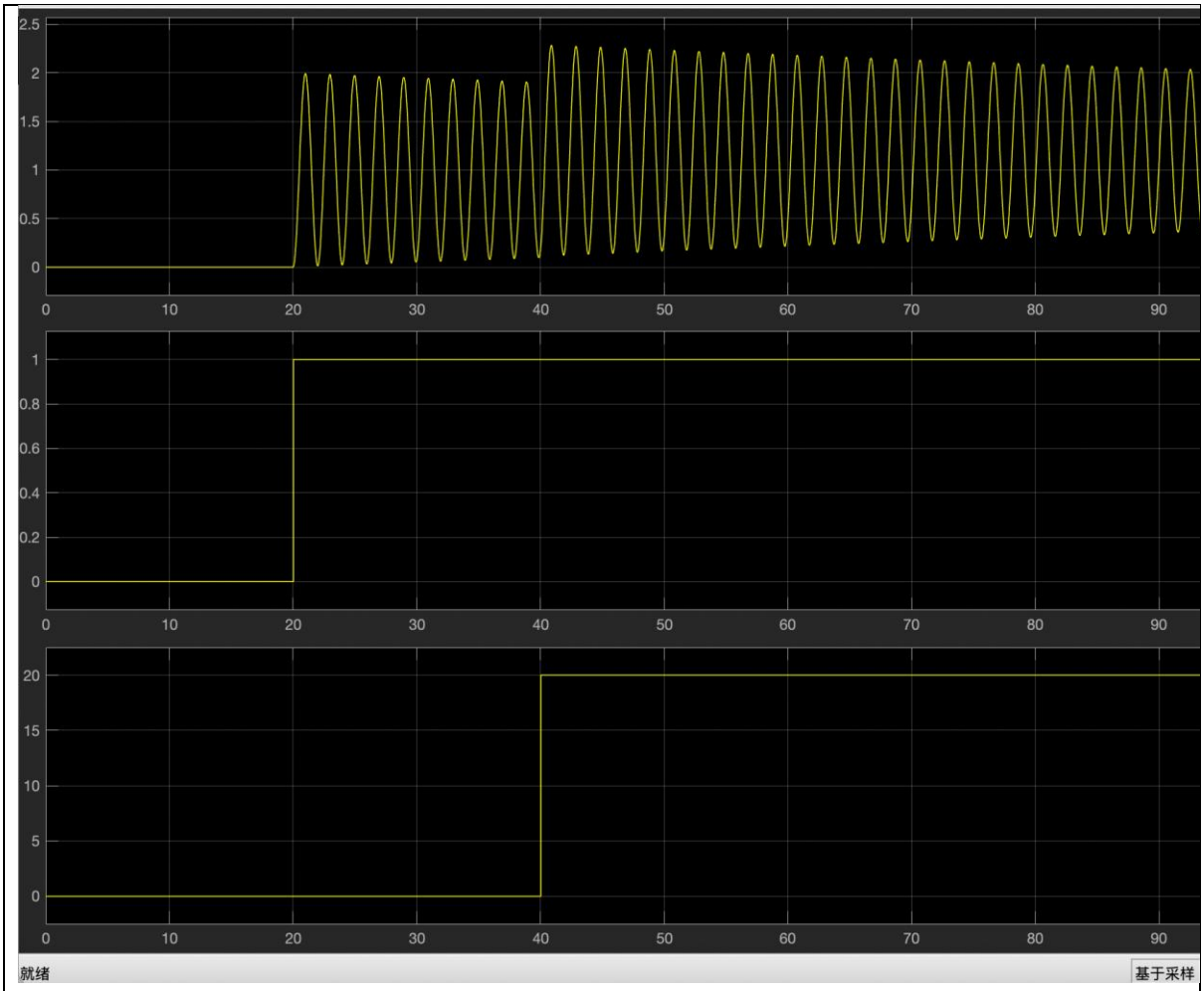


Figure 13

Increasing K_p to 100:

Reduces the steady-state error significantly, since the controller reacts more strongly to error.

However, it does not eliminate the error completely, because the system is still Type 0 (no integrator in the loop).

The transient response becomes faster (shorter rise time), possibly with slight overshoot.

Larger K_p improves the performance, but is not sufficient to eliminate steady-state error or reject disturbances.

○ What change in the plant transfer function would need to take place to change this behaviour?

$$\frac{Kp}{10s^2 + 0.1s + Kp}$$

11.Let us now start to calculate the optimal gains for this system. Let us set the integral gain, Ki to 0. This can be added at the end to remove any steady state error. Let us select optimal Kp and Kd gains. Let's assume we want an overall critically damped system with ζ = 1 and let us select a natural frequency,
ω_n of 10 rad/sec.

○ Firstly, calculate the Closed Loop Characteristic Equation. This isn't too difficult as we know the plant – in some cases the plant is unknown, and the controller would have to be designed using the Zeigler Nichols method.

○ Then equating the coefficients calculate the required Kp and Kd to provide the required specification as above.

○ Using the values you have calculated simulate your system in Simulink and then try some experiments by selecting new values of ω_n (and hence Kd and Kp) and note how this changes the response of the output.

○ Comment on the steady state error when using only a PD controller (Kp and Kd gain only). Can it be completely removed only using PD control?

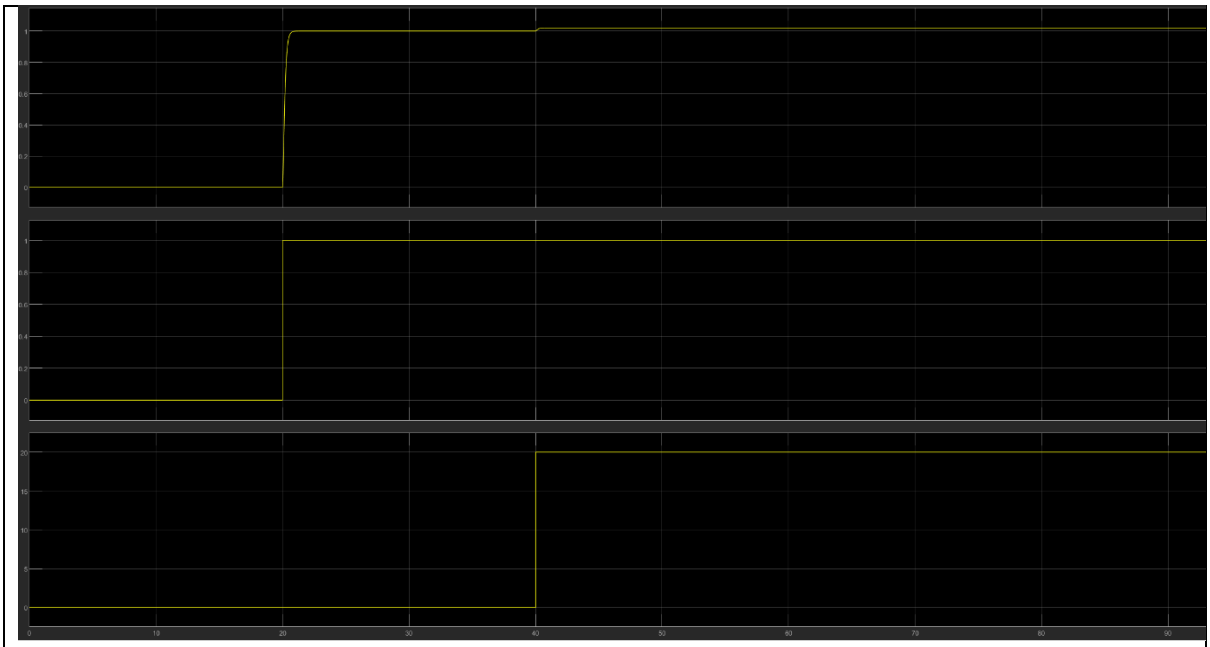


Figure 14

$$\frac{C(s)}{R(s)} = \frac{Kp + sKd}{10s^2 + (0.1 + Kd)s + Kp}$$

$\omega_n = \sqrt{\frac{Kp}{10}}$ and $\zeta = \frac{0.1+Kd}{20} \times \sqrt{\frac{10}{Kp}}$

In order to get ζ as 1 and ωn as 10, then Kp = 1000 and Kd = 199.9

$$c(\infty) = \lim_{s\to 0} s \times \frac{1}{s} \times \frac{Kp + sKd}{10s^2 + (0.1 + Kd)s + Kp} = \frac{Kp}{Kp} = 1$$

as a result, the steady state error is 0.

○ Now introduce integral control, Ki. Calculate the new closed loop Characteristic equation. Note that the integrator raises the order of the system. What is the best method for calculating the maximum allowable gain, Ki, before instability occurs?

Use the Routh–Hurwitz criterion, The characteristic equation turns to $s^3 + 20s^2 + 100s + \frac{Ki}{10} = 0$, Ki needs to be less than 20000.

Ki = 18000:

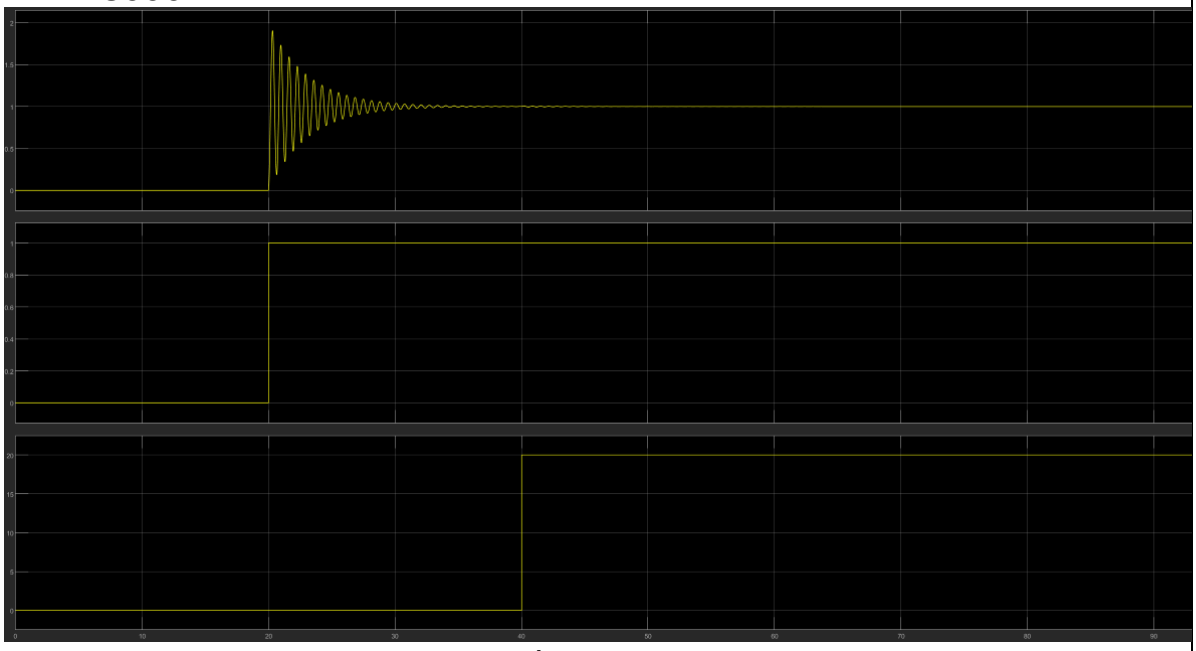


Figure 15

Ki = 21220:

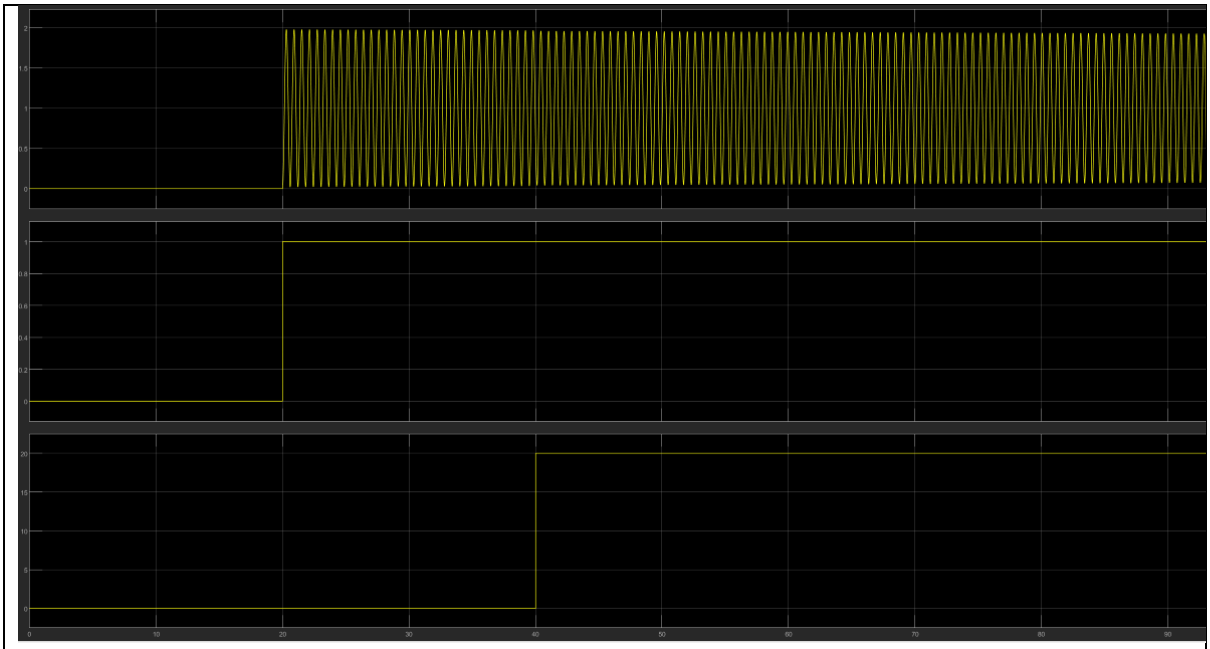


Figure 16

Ki = 25000:

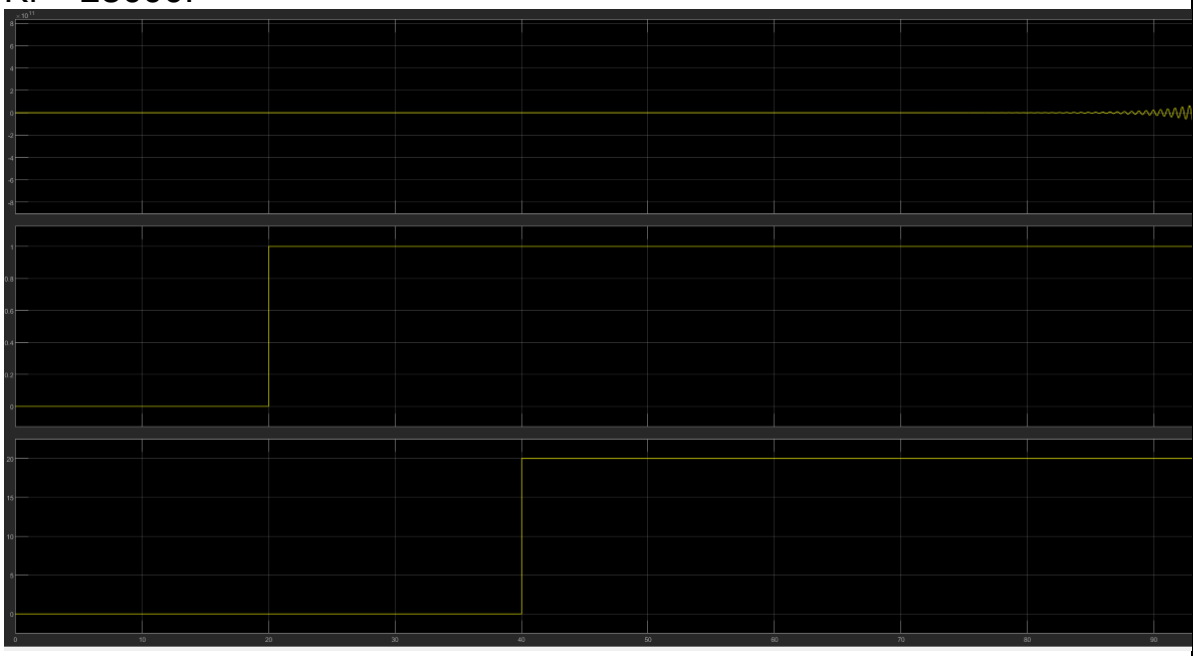


Figure 17

- Use this method and simulate with the calculated gain values in Simulink to verify your calculation.
- Observe any ringing that occurs during the disturbance – does the frequency match your target ω_n?

K_p= 4000, K_d= 399.9, K_i = 160000

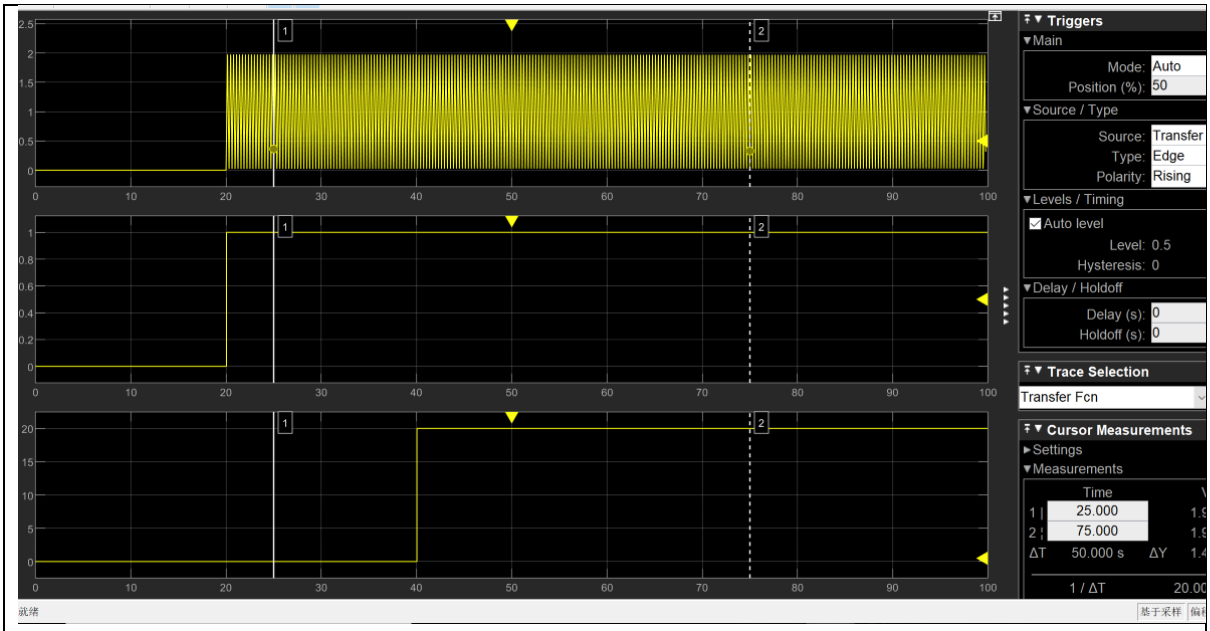


Figure 18

From simulation result, we observe clear ringing in the output signal after the disturbance is applied at around t = 40s.

The frequency measured from the Scope isΔT=50s for 1000 cycles.

So, f=20Hz and ω=2$\pi f \approx$125.66 rad/s

Compared to your target natural frequency ω_n=10 rad/s, the actual oscillation frequency is much higher, indicating that the ringing does not match the target frequency, and suggests that:

The gain values, especially high K_i =160000, are causing the system to enter a high-frequency oscillatory mode

The system is likely overcompensating and generating undesired high-frequency dynamics, possibly on the verge of instability

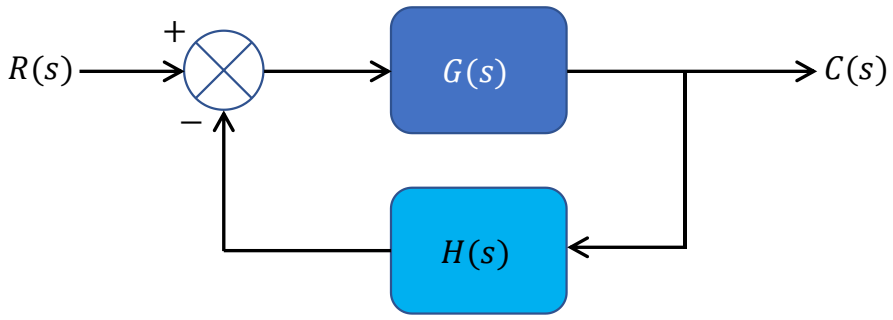
- Does the output respond faster or slower as you increase ω_n?

Increasing ω_n makes the system faster, with reduced rise and peak time, but may increase overshoot or cause minor oscillations.

5.0 Lab 4

Exercise 1

Consider the system:



with:

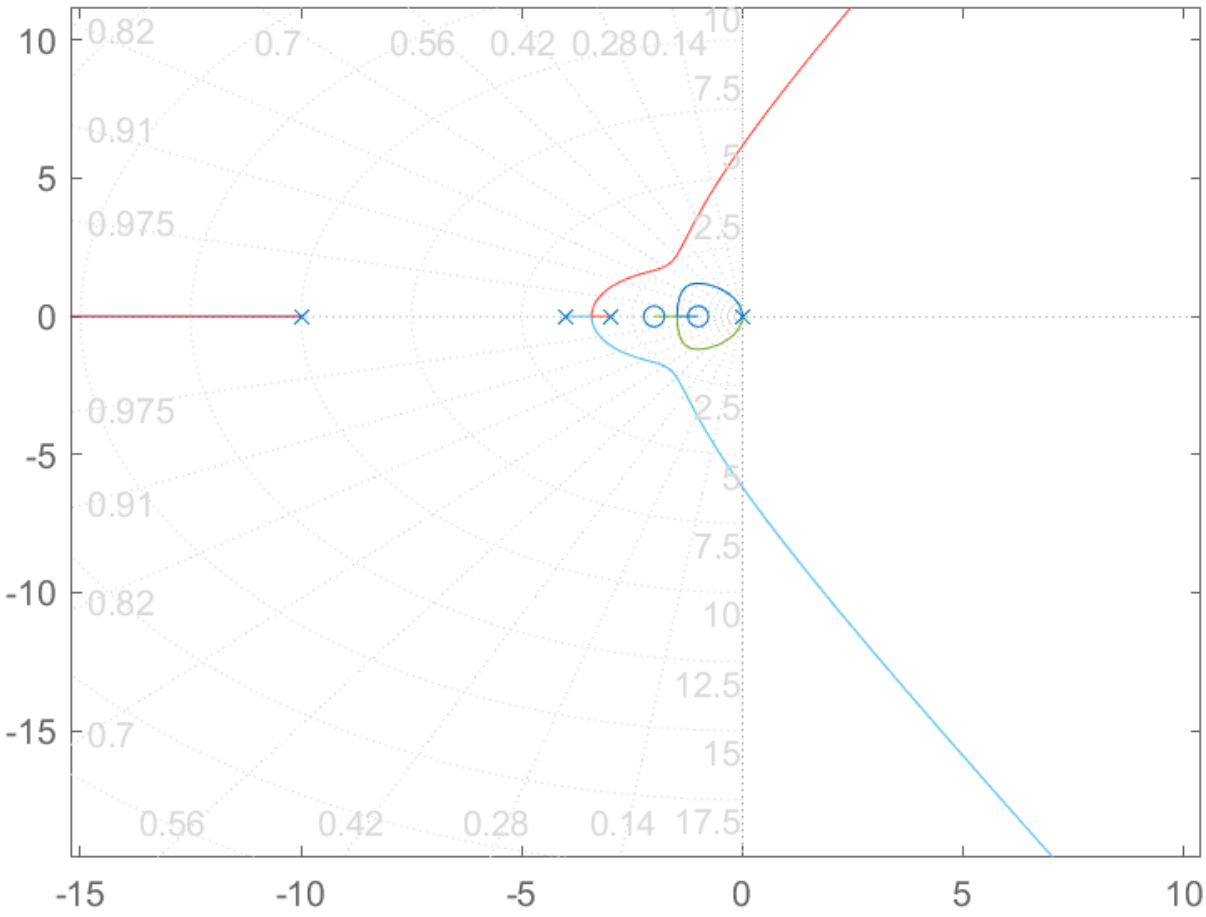
$$G(s) = \frac{(s+1)(s+2)}{s^2(s+3)(s+4)(s+10)}$$

and

$$H(S) = 1$$

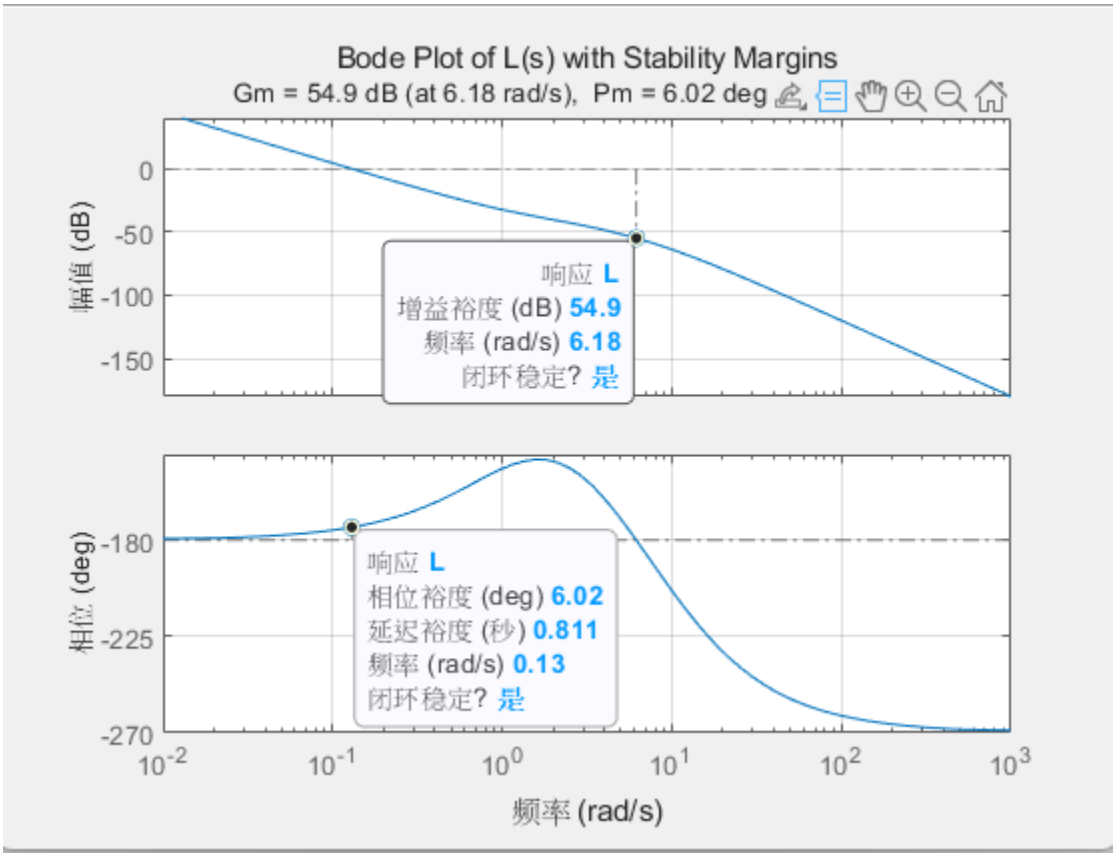
OLTF	$L(s) = G(s)H(S) = \frac{(s+1)(s+2)}{s^2(s+3)(s+4)(s+10)}$
CLTF	$CLTF(s) = \frac{G(s)}{1+G(s)H(S)} = \frac{(s+1)(s+2)}{(s+1)(s+2) + s^2(s+3)(s+4)(s+10)}$

1. Sketch the root locus for the system OLTF and insert the annotated plot here indicating the essential characteristics of a root locus (poles/zeros and their values, breakaway point if any, complex axis crossing,..)



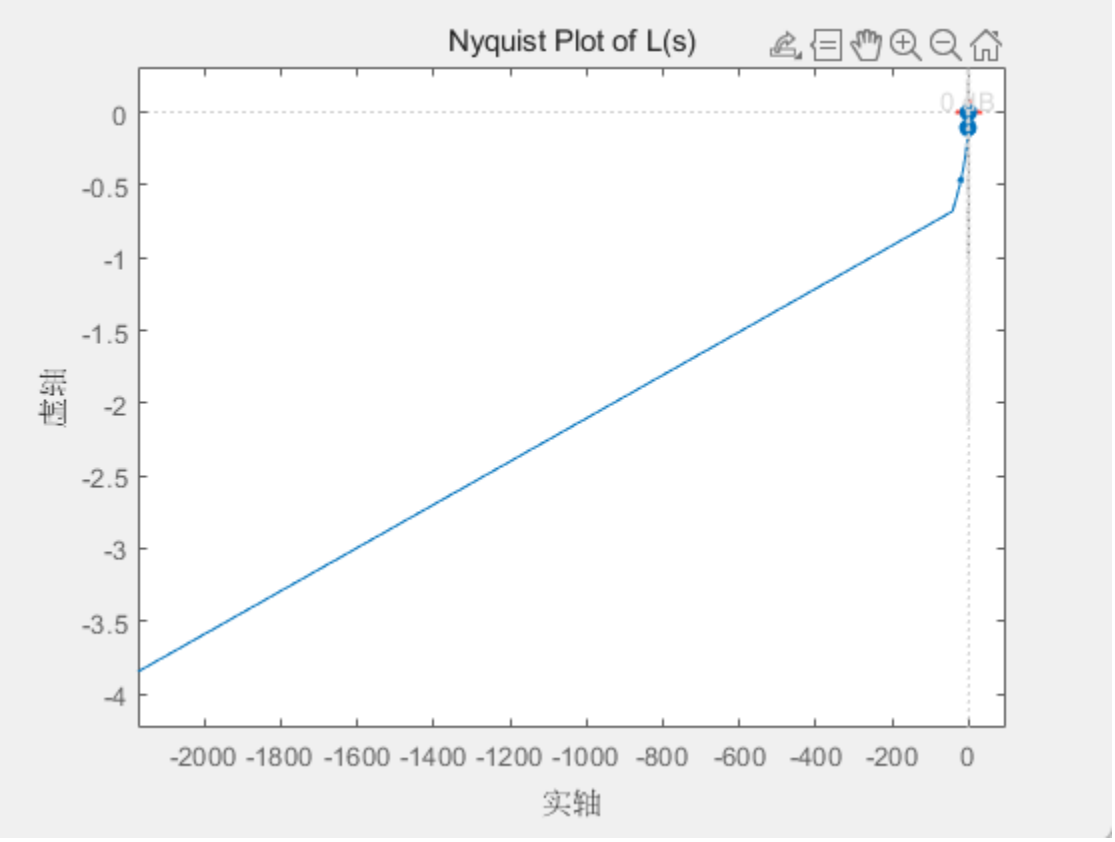
2.Draw the Bode magnitude and phase plots for the system OLTF

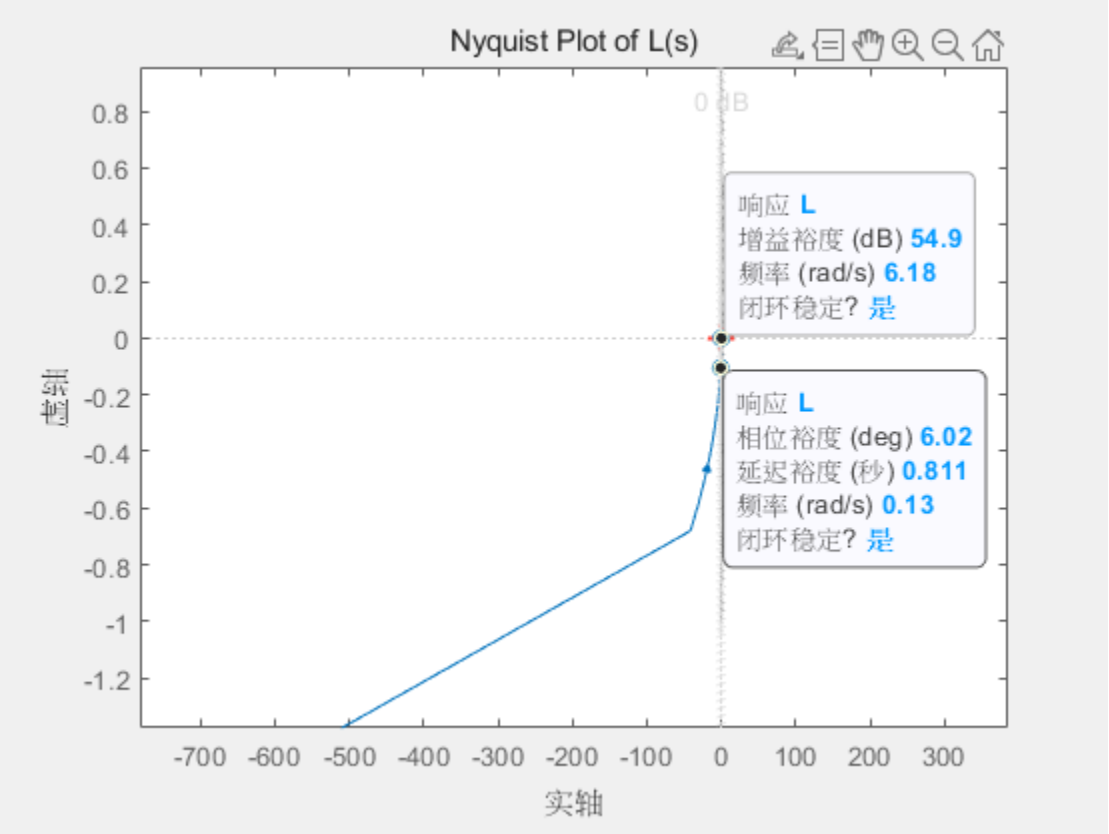
a. Enter the command 'figure'
b. Use the function bode(G) to plot the log-magnitude and phase plots
c. Right click in the graph, go down to the menu 'characteristics' and then select 'All stability margins'
d. Use a datatip to read the values
e. Insert your annotate Bode plot here



3.Also obtain the Nyquist plot for the same system OLTF

a. Enter the command 'figure'
b. Use the function nyquist(G) to obtain the Nyquist plot
c. You may notice that the plot has 2 arrows and in the lectures we've seen that it only goes from 0 to infinity so how come?
d. Well Matlab considers negative frequencies
e. If you right click in the graph, go down to the menu 'Show' and untick 'Negative frequencies'
f. Right click in the graph, go down to the menu 'Characteristics and then select 'All stability margins'
g. Use a datatip to read the values
h. Alternatively you can use the Matlab command window *allmargin(TF)*





GainMargin = 54.93 dB

GMFrequency = 6.18 rad/s

PhaseMargin= 6.02°

PMFrequency= 0.13 rad/s

DelayMargin= 0.8107s

DMFrequency= 0.13 rad/s

Stable/Unstable stable

6.0 Conclusions and summary

The experiments conclusively demonstrated that the damping ratio ζ governs the qualitative form of an RLC circuit's transient response: ζ<1 yields oscillatory underdamped behavior, ζ=1 produces the fastest monotonic response without overshoot, and ζ>1 leads to a slower overdamped response, consistent with textbook predictions. Systematic variations in R, L, and C validated that increasing resistance or capacitance raises ζ (reducing oscillation amplitude and speeding decay), while increasing inductance lowers the natural frequency ω_n (lengthening the time-scale). Closed-loop feedback (Lab 2) was shown to decrease rise and settling times but may introduce overshoot; proportional gain alone reduces steady-state error only partially, whereas the addition of integral action in PID control (Lab 3) eliminates residual error and derivative action improves damping. Simulink simulations reinforced these findings and highlighted trade-offs between speed, stability, and accuracy in controller tuning. Frequency-domain analysis (Lab 4) provided complementary insight into gain and phase margins, confirming system robustness and the impact of controller gains on pole-zero movements. Overall, this lab series has deepened our understanding of dynamic system behavior, control design methodologies, and the effective use of MATLAB/Simulink tools for analysis and implementation.